

一滤皆准：用于状态估计的通用滤波器

Shiqi Liu^{1*}, Wenhan Cao^{1*}, Chang Liu², Zeyu He¹, Tianyi Zhang¹, Yinuo Wang¹, Shengbo Eben Li^{1,3}

¹ 清华大学车辆与运载学院

² 北京大学工学院 ³ 清华大学人工智能学院

* 共同一作 † 通讯作者

摘要

在动力系统中估计隐藏状态，也称为最优滤波，是科学与工程多个领域长期存在的问题。本文提出一种通用滤波框架 **LLM-Filter**¹，它通过把带噪观测嵌入为文本原型，利用大语言模型（LLM）进行状态估计。我们在多种经典动力系统实验中发现：第一，状态估计能够显著受益于预训练 LLM 中蕴含的推理知识。通过与冻结 LLM 建立适当的模态对齐，LLM-Filter 超越了最先进的学习型方法。第二，我们精心设计了提示结构 System-as-Prompt (SaP)，把任务指令纳入其中，使 LLM 能够理解估计任务。在这些提示引导下，LLM-Filter 表现出出色的泛化能力，能够在变化的甚至未见过的环境中准确完成滤波任务。我们进一步观察到 LLM-Filter 中存在 scaling-law 行为：模型规模更大、训练时间更长时，准确率会提高。这些发现使 LLM-Filter 有望成为滤波领域的基础模型。

1 引言

动力系统状态估计是多个领域中的关键主题，包括机器人学 [1]、气象学 [2]、化学 [3] 和交通运输 [4]。状态估计最完整的框架是贝叶斯滤波 [5]，它通过反复执行预测和更新两个步骤来进行在线估计。常用的在线贝叶斯滤波器可分为高斯滤波器 [6] 和粒子滤波器 (PF) [7]。在高维非高斯系统中，高斯滤波器往往会产生显著误差 [8, 9]，而 PF 受到巨大计算需求的限制 [10, 11]。

为了解决这些限制，学习型滤波方法正在形成趋势 [12, 11, 13, 14]。这些学习型滤波器在精心整理的数据集上离线训练，直接捕捉准确的系统统计规律，而不是依赖人工建模，从而实现高准确率和高效在线估计。然而，这些滤波器通常面向特定任务定制；当应用到不同动力系统时，需要重新训练，如图 1 所示。如果不重新训练，当系统发生变化或转向全新环境时，它们的性能通常会退化。

控制领域在适应多样任务和环境时面临类似挑战 [15, 16]。为解决这些挑战，研究者提出了大控制模型，例如 RT [17, 18] 和 OpenVLA [19]。通过利用大语言模型 (LLM) 和视觉-语言模型 (VLM) 的大规模预训练知识，这些大控制模型在多样机器人、任务和环境之间展现出有效泛化能力 [20]。值得注意的是，控制在本质上是滤波的对偶问题；这一点最早通过线性系统中的拉格朗日对偶性得到展示 [21, 22]，后来被扩展到非线性系统 [23, 24]。在滤波领域，据作者所知，目前还没有滤波方法整合 LLM 来利用预训练知识，也尚未形成通用滤波模型。

为填补这一研究空白，我们提出 LLM-Filter：一种在保持主干模型完整性的同时，把 LLM 重编程用于状态估计的通用滤波器。该过程首先把带噪观测嵌入为文本原型，随后由冻结的 LLM 处理，以协调不同数据模态。为实现跨系统泛化，我们用 System-as-Prompt (SaP) 扩展输入，其中包含任务特定指令和任务示例。随后，把 LLM 生成的输出映射到最终状态估计。通过利用 LLM 的预训练知识，并用精心设计的提示加以引导，LLM-Filter 可以超越多数最先进的学习型滤波器，并在完全新系统上表现出显著的上下文内泛化能力；实验对此进行了展示。本文主要贡献如下：

¹代码和数据集可公开访问：<https://github.com/Anonymous-User-2367/LLM-Filter>。

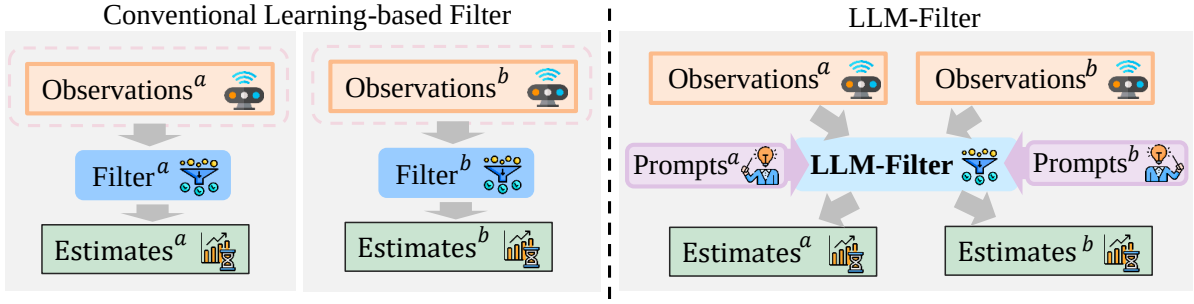


图 1: LLM-Filter 与传统学习型滤波器的比较 左图展示传统学习型滤波器，它依赖特定系统的数据和训练。右图展示 LLM-Filter，它通过提示引导在不同系统之间泛化，而不需要重新训练。

- 我们证明状态估计可以显著受益于预训练语言模型中蕴含的知识。通过在滤波过程与 LLM 推理机制之间建立适当的模态对齐，我们提出了名为 LLM-Filter 的通用滤波器。
- 我们精心设计了一种上下文内方法 SaP，帮助 LLM 理解当前任务并适应特定应用系统。在 SaP 引导下，LLM-Filter 表现出卓越泛化能力，能够在未见场景中有效执行滤波任务。
- 在实验中，LLM-Filter 持续超越最先进的学习型滤波器，并在变化或完全新系统中无需重新训练或微调即可展示显著的上下文内泛化能力。此外，我们观察到 scaling-law 行为，即准确率会随模型规模增大和训练时间延长而提高。

2 预备知识

2.1 状态估计的形式化

我们首先简要回顾状态估计以及本文考虑的主要滤波方法。通常，状态估计可用马尔可夫状态空间模型描述：

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t) + \boldsymbol{\xi}_t, \quad (2.1a)$$

$$\mathbf{y}_t = h(\mathbf{x}_t) + \boldsymbol{\zeta}_t, \quad (2.1b)$$

其中 $t \in \{1, 2, \dots\}$ 。这里 $f(\cdot)$ 和 $h(\cdot)$ 分别表示状态转移模型和观测模型。 $\mathbf{x}_t \in \mathbb{R}^M$ 定义为状态， $\mathbf{y}_t \in \mathbb{R}^N$ 表示观测。此外， $\boldsymbol{\xi}_t$ 和 $\boldsymbol{\zeta}_t$ 分别表示转移噪声和观测噪声，通常假设它们的概率分布已知。式 (2.1) 中的状态空间模型描述可以表示为隐马尔可夫模型：

$$\begin{aligned} \mathbf{x}_0 &\sim p(\mathbf{x}_0), \\ \mathbf{x}_t &\sim p(\mathbf{x}_t | \mathbf{x}_{t-1}), \\ \mathbf{y}_t &\sim p(\mathbf{y}_t | \mathbf{x}_t), \end{aligned} \quad (2.2)$$

其中 $p(\mathbf{x}_t | \mathbf{x}_{t-1})$ 和 $p(\mathbf{y}_t | \mathbf{x}_t)$ 分别是转移概率与输出概率， $p(\mathbf{x}_0)$ 表示初始状态分布。从本质上看，(2.1) 和 (2.2) 是同一底层系统模型的不同表述。状态估计的目标，是在所有可用观测 $\mathbf{y}_{1:t}$ 上提取状态 $\mathbf{x}_t$ 的信息。

2.2 在线贝叶斯滤波

状态估计的一个原则性框架是贝叶斯滤波，它通过两个步骤递归计算 $p(\mathbf{x}_t | \mathbf{y}_{1:t})$ ：

$$p(\mathbf{x}_t | \mathbf{y}_{1:t-1}) = \int p(\mathbf{x}_t | \mathbf{x}_{t-1}) p(\mathbf{x}_{t-1} | \mathbf{y}_{1:t-1}) d\mathbf{x}_{t-1}, \quad (2.3a)$$

$$p(\mathbf{x}_t | \mathbf{y}_{1:t}) = \frac{p(\mathbf{y}_t | \mathbf{x}_t) p(\mathbf{x}_t | \mathbf{y}_{1:t-1})}{\int p(\mathbf{y}_t | \mathbf{x}_t) p(\mathbf{x}_t | \mathbf{y}_{1:t-1}) d\mathbf{x}_t}, \quad (2.3b)$$

其中 (2.3a) 称为预测步骤, (2.3b) 称为更新步骤。在线性和高斯噪声假设下, (2.3) 可以在线解析计算, 这正是卡尔曼滤波 (KF) [25] 的基础, 详见附录 A。然而, 这些滤波方法高度依赖对各系统特定的 $f(\cdot)$ 、 $h(\cdot)$ 以及 ξ_t 与 ζ_t 噪声分布的准确建模。

2.3 学习型滤波

与 (2.3) 中的在线计算不同, 学习型滤波方法在精心收集的数据上训练, 学习观测与底层状态之间的相关关系。这一学习通过优化如下监督学习损失来更新网络参数 θ :

$$\mathcal{L}(\theta) = \|\mathbf{x}_t - \hat{\mathbf{x}}_t(\mathbf{y}_{1:t}, \theta)\|_2^2, \quad (2.4)$$

其中 $\hat{\mathbf{x}}_t(\mathbf{y}_{1:t}, \theta)$ 表示估计值, $\mathbf{x}_t$ 是真实值。通过从训练数据中捕获准确系统统计规律, 而不是依赖人工建模, 学习型滤波方法通常能够提供更精确的估计和更高效率。然而, 对任务特定数据集的强依赖使传统学习型滤波器缺乏灵活性; 只要任务发生变化或转向全新环境, 就需要重新训练。为克服这一限制, 本文旨在提出一种利用上下文内机制的通用滤波方法, 使其无需重新训练即可在多个系统中有效工作。

3 方法

LLM-Filter 的主要结构如图 2 所示。我们首先在 LLM 的离散 token 与连续观测之间进行模态对齐 (第 3.1 节), 然后基于系统信息设计一种上下文内推理方法以增强泛化能力 (第 3.2 节), 最后从 LLM 预测 token 中获得估计值并定义损失函数 (第 3.3 节)。

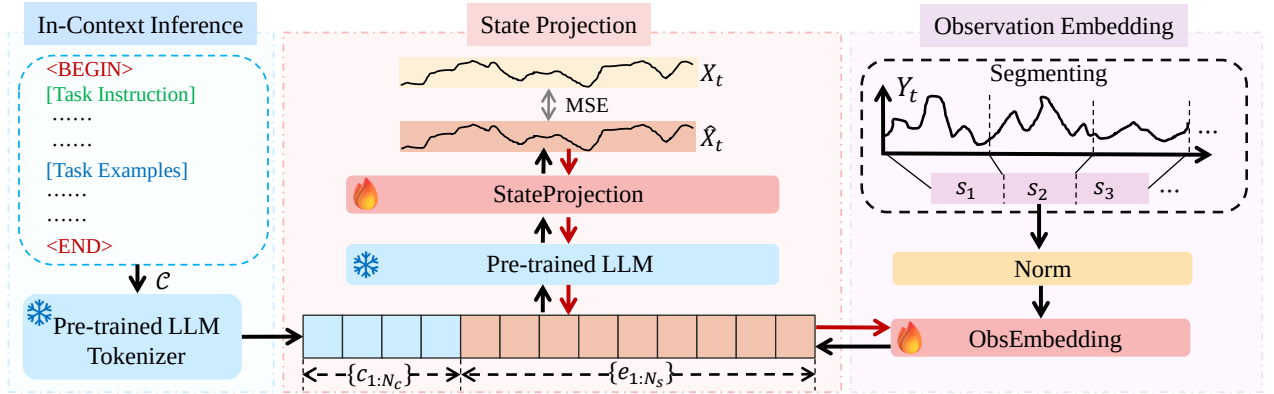


图 2: LLM-Filter 框架 (1) 观测嵌入: 观测被分段并嵌入为 token embedding。(2) 上下文内推理: 由 tokenized SaP 上下文得到的上下文 token 与观测 embedding 拼接。(3) 状态投影: LLM 预测的 token 被投影到状态空间, 以获得最终估计。

3.1 观测嵌入

移动时域 如第 2.1 节所述, 目标是在所有可用观测 $\mathbf{y}_{1:t}$ 上估计状态 $\mathbf{x}_t$ 。然而, 把不断增长的观测序列全部输入是不现实的。受移动时域估计 [26] 和 LLM 上下文窗口 [27] 启发, 我们设计了一个固定长度为 T 的滑动窗口用于估计。输入观测定义为 $\mathbf{Y}_t = \{\mathbf{y}_{t-T+1:t}\} \in \mathbb{R}^{T \times N}$, 对应输出估计为 $\hat{\mathbf{X}}_t = \{\hat{\mathbf{x}}_{t:t+T-1}\} \in \mathbb{R}^{T \times M}$, 其中 $\mathbf{X}_t = \{\mathbf{x}_{t:t+T-1}\} \in \mathbb{R}^{T \times M}$ 表示真实状态, N 和 M 分别表示观测变量与状态变量维度。

分段与嵌入 为把连续观测与离散 token 对齐, 我们将输入观测分段并嵌入到 LLM 中。一种常见分段方法使用 单序列格式 [28, 29], 即直接展平所有数据维度。然而, 这种方式可能破坏变量之间的内在相关性, 例如位置和速度之间对准确位置估计至关重要的关系。为保留内在相关性, 我们按分段长度 L 对 $\mathbf{Y}_t$ 进行分段, 同时保留其多维结构:

$$\mathbf{Y}_t = \{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_{N_s}\}, \quad N_s = \left\lfloor \frac{T}{L} \right\rfloor,$$

其中每个分段 $\mathbf{s}_i \in \mathbb{R}^{L \times N}$ ($i = 1, 2, \dots, N_s$), $\lfloor \cdot \rfloor$ 表示向下取整函数。若不能整除, 则在末尾填充以确保所有分段长度均为 L 。为了利用 LLM 的预训练知识和内在 token 转移能力, 我们冻结参数, 并去除为语言 token 设计的嵌入层和投影层。相反, 我们引入 $\text{ObsEmbedding}(\cdot) : \mathbb{R}^{L \times N} \mapsto \mathbb{R}^{D \times N}$, 它把每个归一化分段独立嵌入到 LLM 的 D 维潜空间:

$$\mathbf{e}_i = \text{ObsEmbedding}(\text{Norm}(\mathbf{s}_i)), \quad i = 1, 2, \dots, N_s.$$

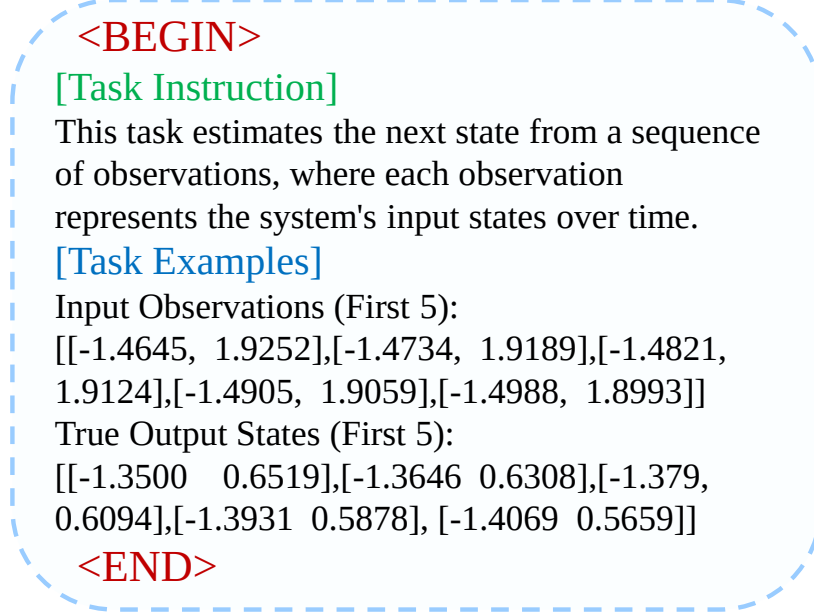


图 3: Hopf [30] 系统中 SaP 的示意。

3.2 上下文内推理

传统学习型滤波器仅从观测数据估计真实系统状态, 如第 2.3 节所述。然而, 这种依赖限制了它们识别底层系统动力学的能力, 导致其对新系统泛化较差。为解决这一点, 我们基于 LLM 的上下文内学习能力 [31], 提出一种新的提示策略 SaP, 用于引导 LLM-Filter 灵活适应不同系统。SaP 建立在两个对构造 LLM-Filter 有效提示必需的组成部分之上: 任务指令和任务示例。**任务指令**为 LLM-Filter 提供关键上下文知识, 这些知识可能因不同领域而异。**任务示例**通过展示样例场景来澄清当前任务, 从而引导 LLM-Filter。SaP 的示例见图 3。推理时, SaP 文本 $\mathcal{C}$ 会被输入预训练 LLM tokenizer 以处理语言文本:

$$\{\mathbf{c}_1, \dots, \mathbf{c}_{N_c}\} = \text{LLMTokenizer}(\mathcal{C}),$$

其中 N_c 是 tokenized 上下文 token 的数量。

3.3 状态投影

Token 预测 在多样下游任务上训练的主流 LLM, 擅长基于已有 token 在多个领域中预测下一个 token [32, 33]。通过输入引导性 SaP 和前一观测的 embedding token, 我们希望得到下一时段的状态估计特征。为实现模态对齐, 我们只使用 LLM 的核心层, 排除原本用于语言 token 的嵌入层和投影层。tokenized SaP 上下文与观测 embedding $\{\mathbf{c}_1, \dots, \mathbf{c}_{N_c}, \mathbf{e}_1, \dots, \mathbf{e}_{N_s}\}$ 被直接输入核心层, 核心层随后生成对应输出 token embedding $\{\hat{\mathbf{e}}_1, \dots, \hat{\mathbf{e}}_{N_s}\}$:

$$\begin{aligned} \hat{E} &= \text{LLMCoreLayers}(\{\mathbf{c}_1, \dots, \mathbf{c}_{N_c}, \mathbf{e}_1, \dots, \mathbf{e}_{N_s}\}), \\ \{\hat{\mathbf{e}}_1, \dots, \hat{\mathbf{e}}_{N_s}\} &= \hat{E}_{\{N_c+1:N_c+N_s\}}. \end{aligned} \tag{3.1}$$

输出投影 随后, 我们使用 $\text{StateProjection}(\cdot) : \mathbb{R}^{D \times N} \mapsto \mathbb{R}^{T \times M}$ 层, 把输出 embedding $\{\hat{\mathbf{e}}_1, \dots, \hat{\mathbf{e}}_{N_s}\}$ 投影到系统状态空间, 由此获得最终估计 $\hat{\mathbf{X}}_t$:

$$\hat{\mathbf{X}}_t = \text{StateProjection}(\{\hat{\mathbf{e}}_1, \dots, \hat{\mathbf{e}}_{N_s}\}),$$

其中 $\text{ObsEmbedding}(\cdot)$ 和 $\text{StateProjection}(\cdot)$ 均由简单 MLP 实现, 这种方式已被证明可以有效嵌入并投影面向 LLM 的时间序列 [28]。

损失函数 最后, 总体目标是最小化真实值 $\mathbf{X}_t$ 与输出估计 $\hat{\mathbf{X}}_t$ 之间的误差, 以优化 LLM-Filter $\mathcal{F}_\theta$ 的参数 θ :

$$\mathcal{L}_\theta = \frac{1}{T} \|\mathbf{X}_t - \hat{\mathbf{X}}_t\|_2^2 = \frac{1}{T} \|\mathbf{X}_t - \mathcal{F}_\theta(\mathbf{Y}_t, \mathcal{C})\|_2^2.$$

这一损失函数可视为 (2.4) 的扩展, 因为它纳入了额外上下文输入。值得注意的是, 为保持预训练知识并受资源约束, 我们冻结 LLM, 仅更新 $\text{ObsEmbedding}(\cdot)$ 和 StateProjection 的参数。LLM-Filter 的详细配置见附录 D.2。

说明 1. 当 SaP 提示不可用时, LLM-Filter 仍可在无上下文引导的情况下运行; 我们称这一变体为 **LLM-Filter-O**。然而, 由于缺乏上下文引导, 它的滤波准确率与泛化能力可能下降, 第 5 节的比较展示了这一点。

4 相关工作

在线贝叶斯滤波 状态估计领域可追溯到 KF [25] 的开创性发展; KF 曾著名地应用于 NASA Ames Research Center 的 Apollo 制导与导航系统 [34]。这一里程碑推动了扩展卡尔曼滤波 (EKF) [35]、无迹卡尔曼滤波 (UKF) [36] 等高级方法, 使其能够应用于非线性系统。然而, 它们对线性化和高斯假设的依赖会在复杂场景中引入显著误差。作为回应, 粒子型滤波方法被提出, 包括 PF [37] 和集合卡尔曼滤波 (EnKF) [38]。遗憾的是, 粒子型方法面临高计算需求和维度灾难。

学习型滤波 神经网络融合的近期进展推动了学习型滤波方法的发展。例如, KalmanNet [11, 39] 采用 RNN 计算 Kalman 增益, 显著提升计算效率和准确率。类似地, MEstimator [13] 和 RStateNet [14] 使用多层感知机 (MLP) 和长短期记忆网络 (LSTM) [40] 处理历史状态 ensemble, 从而丰富输入信息并获得更高精度。另一方面, ProTran [41] 使用注意力机制在潜空间中建模系统动力学, 实现长期估计。然而, 这些方法仅依赖观测数据作为输入, 当应用到新系统时需要重新训练。相比之下, 我们提出的 LLM-Filter 对齐语言与状态估计的模式, 使其能够通过上下文内引导泛化到未见系统, 而无需额外训练。

大控制模型 作为状态估计的对偶问题, 控制领域越来越倾向于在跨 embodiment 的大规模多样数据集上训练多任务通用控制模型。近期工作已把 LLM 和 VLM 纳入控制 [15, 32], 但主要聚焦高层规划, 未充分利用大规模模型中的知识。为此, RT 模型 [17, 18, 20] 直接训练面向开放词表视觉问答的 VLM 输出低层机器人动作。在这一架构基础上, OpenVLA [19] 简化了模型参数, 同时提升性能, 并探索进一步微调用于泛化的策略。大控制模型的快速进展推动我们开发一种能够利用预训练 LLM 知识的通用状态估计模型。

用于时间序列预测的 LLM 状态估计和时间序列预测都使用序列数据。然而, 状态估计从带噪观测中估计隐藏状态, 而时间序列预测则从同一数据预测未来值。近期研究已经取得显著突破, 尤其是在 zero-shot 预测中, 通过建立 LLM token 与数值数据之间的映射来实现 [42, 43]。此外, TimeLLM [44] 探索了 LLM 的上下文内预测能力, 利用上下文内学习引入多种预测任务。AutoTimes [28] 展示了时间戳编码技术如何进一步释放 LLM 在时间序列应用中的潜力。虽然这些模型主要基于过去数据预测未来值, LLM-Filter 则聚焦于从动态系统中的带噪观测估计隐藏状态。

5 实验

我们的实验评估旨在回答以下问题:

1. LLM-Filter 能否在规范估计任务中学会滤波? (第 5.1 节)
2. LLM-Filter 能否有效泛化到新系统? (第 5.2 节)
3. LLM 的内在特征是否会在 LLM-Filter 中显现? (第 5.3 节)

Benchmark 为全面评估 LLM-Filter 的性能, 我们在两类经典动力系统上进行了实验: (1) 低维非线性系统; (2) 高维混沌系统。这些系统常被先前研究用作 benchmark [45, 46, 2]。具体而言, 低维非线性系统包括 Selkov [47]、Oscillator [48]、Hopf [30] 和 Double Pendulum [49] 系统。对于高维混沌系统, 我们在 Lorenz96 [50] 和 VL20 [51] 上进行实验; 二者都广泛用于大气动力学研究。对于泛化实验, 我们还纳入 Tracking 系统 [5]。各系统的详细描述和配置见附录 B。我们使用均方根误差 (RMSE) 和平均运行时间 (ms/step) 作为主要评价指标, 其精确定义见附录 D.3。

Baselines 我们将 LLM-Filter 与最先进方法进行比较, 包括在线贝叶斯滤波器: PF [10]、EnKF [38]、average-particle ensemble Kalman filter (EnKFI) [46]、soft ensemble Kalman filter (EnKFS) [46]、huber ensemble Kalman filter (HubEnKF) [52], 以及学习型方法: MEstimator [13]、RStateNet [14]、ProTran [41] 和 KalmanNet [11]。为公平比较, 所有在线贝叶斯滤波器的超参数都在第一次试验中使用 Bayesian Optimization (BO) 包 [53] 选择。这些方法的详细描述见附录 D.1。除非另有说明, LLM-Filter 使用 LLaMA-7B [54] 作为基础 LLM。

5.1 规范估计任务结果

在本实验中, 我们评估 LLM-Filter 与 LLM-Filter-O 在规范估计任务中的基本滤波能力, 并与包括 MEstimator、RStateNet、ProTran、KalmanNet、EnKF 和 PF 在内的最先进方法比较。

低维非线性系统 我们首先在低维非线性系统上进行实验。结果汇总于表 1, 其中 LLM-Filter 在所有测试系统上均持续优于其他方法。值得注意的是, LLM-Filter 的 RMSE 在所有系统上最高降低了 32.00%。平均来看, 它相对于表现最好的学习型滤波方法提升了 17.93%。与表现最好的在线贝叶斯滤波器相比, LLM-Filter 仍取得了平均 21.65% 的提升。此外, 使用上下文提示后, LLM-Filter 相对于只使用数据的 LLM-Filter-O 提升了 10.18%, 说明上下文内提示帮助 LLM-Filter 更好地理解系统并执行更准确的状态估计。这些结果突出体现了 LLM-Filter 在低维非线性系统中的强大滤波能力, 并确立了其 state-of-the-art 性能。

表 1: 经典系统上状态估计的 RMSE。最佳结果以 **粗体** 标出, 第二好结果以下划线标出。标为 *NaN* 的结果表示模型在训练中发散。

方法	学习型滤波器						在线贝叶斯滤波器	
	LLM-Filter	LLM-Filter-O	MEstimator	RStateNet	ProTran	KalmanNet	EnKF	PF
Selkov	0.4061	0.6369	0.8864	0.7202	1.0219	1.1662	<u>0.5978</u>	0.6863
Oscillator	0.5247	0.5753	0.8347	0.8493	0.8933	0.5665	<u>0.5505</u>	0.6807
Hopf	0.5751	0.8180	0.8290	0.7282	0.7146	1.1984	<u>0.6322</u>	0.6801
Pendulum	0.8348	0.9218	0.9354	0.9180	<u>0.8456</u>	2.7140	1.4117	5.3788
Lorenz96	0.9149	0.9735	<u>0.9649</u>	0.9762	0.9975	NaN	6.6024	4.6289
VL20	0.7717	0.8433	1.0014	<u>0.9428</u>	0.9902	NaN	5.8633	11.6980
Average	0.6712	<u>0.7948</u>	0.9086	0.8558	0.9105	NaN	2.6097	3.9588

高维混沌系统 随后, 我们在 Lorenz96 和 VL20 系统上进行实验; 二者均有 72 个状态维度, 用于模拟大气动力学。结果汇总于表 1。LLM-Filter 在 Lorenz96 与 VL20 系统上取得 state-of-the-art 性能, 相对于最佳 baseline 滤波器平均提升 11.7%。相比之下, 基于官方代码库实现的 KalmanNet² 最终在训练期间发散。这些结果验证了 LLM-Filter 能够有效处理高维混沌系统。

²官方开源仓库位于 https://github.com/KalmanNet/KalmanNet_TSP。尽管我们优化了超参数和网络结构, KalmanNet 仍难以过滤高维系统, 潜在原因可能是计算 Kalman 增益时涉及较高维度。

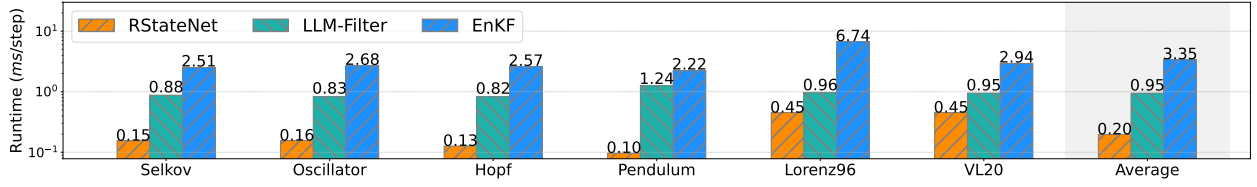


图 4: LLM-Filter、EnKF 与 RStateNet 的估计运行时间比较。完整结果见表 6。

运行时间 运行时间是实际滤波应用中的关键因素。对于学习型滤波器和在线贝叶斯滤波器，我们分别选择 RStateNet 和 EnKF 作为代表方法进行比较。结果汇总于图 4。在所有系统上，LLM-Filter 的运行时间慢于参数更少的 RStateNet。然而，LLM-Filter 快于在线 EnKF，因此是实时应用中非常实际且有效的解决方案。随着系统维度增加，EnKF 和 RStateNet 的计算时间都明显增长。相比之下，LLM-Filter 的运行时间稳定保持在约 $1.0ms$ ，这突出了 LLM-Filter 在高维系统估计中的效率，并为此类应用提供了明确优势。测试设备配置、模型参数和峰值内存的详细信息见附录 D 与表 5。

5.2 估计任务的泛化评估

在本实验中，我们评估所提出 LLM-Filter 的跨适应能力，具体而言，即它在被应用到修改后的系统甚至完全不同系统时的表现。

模型不匹配 模型不匹配是滤波任务中的常见挑战，指假定的观测噪声方差不同于实际噪声方差 [11, 46]。我们在 Selkov、Oscillator 与 Hopf 系统上评估模型不匹配下的 LLM-Filter，并用观测协方差扩展比 (OCER) 量化不匹配严重程度。我们将 LLM-Filter 与针对模型不匹配设计的鲁棒滤波器比较，包括 EnKF、EnKFS 和 HubEnKF。结果如图 5 所示。随着 OCER 增大，鲁棒滤波器性能快速下降，而 LLM-Filter 保持稳定准确率，退化极小。这些发现突出了 LLM-Filter 在变化观测条件下的卓越鲁棒性和适应能力。

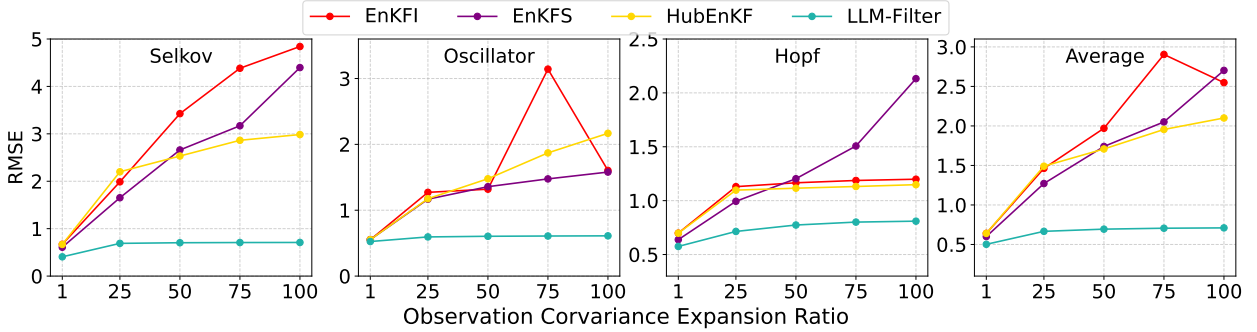


图 5: 随着观测协方差扩展比增大，Selkov、Oscillator 和 Hopf 系统上的 RMSE 结果比较。最右子图给出这些系统上的平均结果。完整结果见表 7。

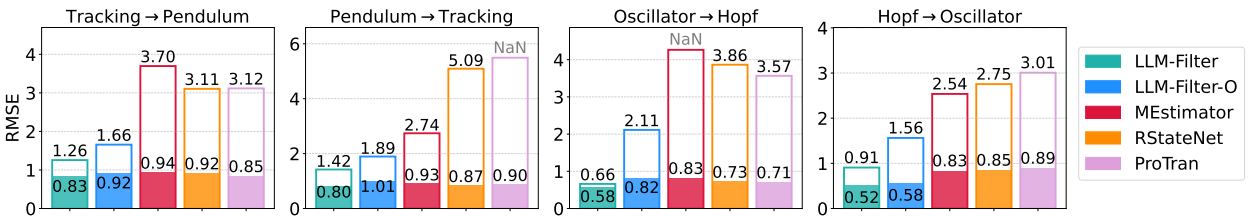


图 6: 跨系统场景的 RMSE。空心柱表示跨系统场景，实心柱对应模型在同一系统上训练和测试的情形。在跨系统场景中，*Tracking* $\rightarrow$ *Pendulum* 表示模型在 *Tracking* 系统上训练，并在 *Pendulum* 系统上评估。完整结果见表 9。

跨系统 在本实验中，我们评估在完全不同系统上的表现，如图 6 所示。跨系统结果用空心柱展示，而实心柱表示直接在目标系统上训练的 baseline 模型性能。我们还纳入 LLM-Filter-O 作为消融比较。在 SaP 提示

引导下, LLM-Filter 取得出色性能, 在跨系统场景中表现出所有方法中最低的 RMSE。与专门训练的 baseline 相比, LLM-Filter 提供了有竞争力的性能, 甚至在 *Oscillator* $\rightarrow$ *Hopf* 场景中取得最佳结果。缺少 SaP 提示引导时, LLM-Filter-O 的泛化能力相较 LLM-Filter 出现明显下降。此外, LLM-Filter 的实心柱和空心柱差异很小, 说明即使应用于此前未见系统, 它也保持了强性能。相比之下, 其他学习型方法性能明显下降, 甚至在面对未见系统时失去提供准确估计的能力。这些结果展示了 LLM-Filter 对新系统有效泛化的潜力。

5.3 对 LLM 的探索

本小节探索 LLM-Filter 中 LLM 的 scaling law 与微调方法。此外, 我们开展 LLM 组件消融研究, 以强调它在模型成功中的关键作用。

Scaling 行为 Scaling laws 是关于性能与效率指标如何随模型规模增加而变化的经验观察 [55]。在本研究中, 我们通过评估不同参数规模的 LLM 主干模型, 探索 LLM-Filter 在状态估计任务中的 scaling 行为。为避免不同模型 tokenizer 差异带来的不一致, 本实验未使用上下文内提示。我们考察了所有低维非线性系统与高维混沌系统上的平均 RMSE 和训练效率, 结果如图 7 所示。结果表明 LLM-Filter 存在 scaling 行为: 随着模型参数增加, RMSE 下降、估计准确率提升, 但代价是训练时间更长。此外, 实验也强调了 LLM-Filter 的适应性, 因为它在多种 LLM base 上都能取得强性能。

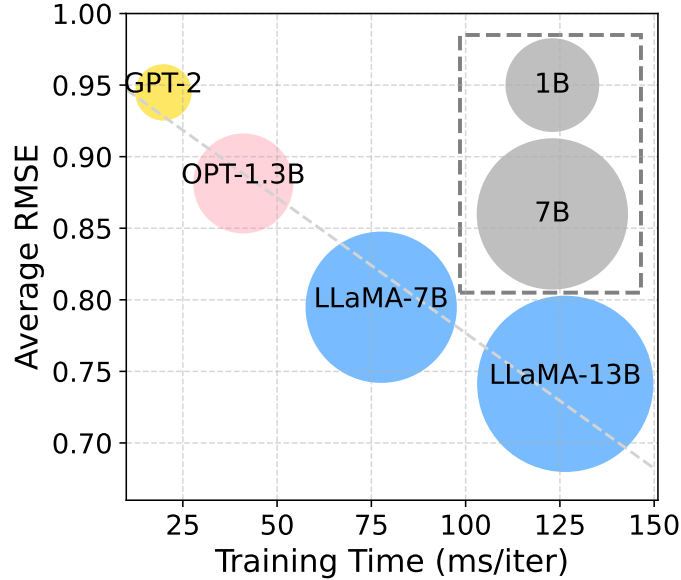


图 7: 不同 LLM 上 RMSE 与效率的比较。完整结果见表 10。

全量微调与 LoRA 本实验考察两种常用于让 LLM 更有效适配下游任务的 prominent 方法: 全参数微调 [56] 和低秩适配 (LoRA) [57]。对于 LLM-Filter, 全量微调会更新所有模型参数, 而 LoRA 只更新模型内部的低秩矩阵。结果汇总于表 2。有趣的是, 为 LLM-Filter 配备 LoRA 导致性能下降。我们认为, 这是因为低秩矩阵调整存在限制, 可能缺少状态估计任务所需高精度的表达能力。另一方面, 全量微调后的 LLM-Filter 在更具挑战性的环境中表现更好, 例如 Lorenz96 和 VL20 系统。但这也带来代价: 在更简单的动态系统中, 性能反而更差, 这可能是大量参数更新造成过拟合所致。

LLM 消融研究 在本实验中, 我们回应一个近期 LLM 时间序列分析研究中提出的问题: 在状态估计任务中, 使用 LLM 是否确实必要 [58]。为研究这一点, 我们把 LLM-Filter 中的 LLM 替换为传统神经网络架构, 包括 MLP、RNN 和 Transformer, 同时保持所有其他组件不变。结果汇总于表 3。结果清楚表明, LLM-Filter 在所有测试系统上持续优于这些传统网络架构。具体而言, LLM-Filter 的 RMSE 相对于 MLP、RNN 和 Transformer 分别平均提升 31.66%、32.43% 和 30.73%。这一显著改进突出了 LLM 在状态估计任务中的优越性能和关键作用, 验证了其实现 state-of-the-art 结果的必要性。

表 2: 使用 LoRA 或全量微调时 LLM-Filter 的 RMSE。

方法	LLM-Filter	+LoRA	+Full Finetune
Selkov	0.4061	<u>0.4498</u>	0.7103
Oscillator	0.5247	<u>0.5370</u>	0.5513
Hopf	0.5751	<u>0.6744</u>	0.8444
Lorenz96	0.9149	<u>0.9147</u>	0.9140
VL20	<u>0.7717</u>	0.8962	0.7027

表 3: LLM-Filter 的 LLM 消融研究 RMSE。

方法	LLM-Filter	MLP	RNN	Transformer
Selkov	0.4061	0.9910	0.9862	<u>0.8863</u>
Oscillator	0.5247	<u>0.8176</u>	0.9481	0.8566
Hopf	0.5751	0.9057	1.0114	<u>0.8610</u>
Lorenz96	0.9149	<u>0.9661</u>	1.0105	0.9721
VL20	0.7717	0.9977	<u>0.8043</u>	0.9598

6 结论与未来工作

受大控制模型成功的启发，我们提出一种通用大滤波模型 LLM-Filter，旨在通过从 LLM 转移知识来解决特定估计任务，包括泛化任务。该过程首先把带噪观测嵌入为文本原型，然后用 SaP 方法丰富输入；SaP 纳入基本任务指令和特征。这些输入随后由冻结 LLM 处理，生成的输出再投影到最终估计。在提示引导和预训练知识支持下，LLM-Filter 超越专用学习型方法，并在多种系统之间展现强泛化能力。

LLM-Filter 的泛化能力目前仍限制在具有相同维度的系统上，其在多样设置中的评估仍然有限。未来工作将聚焦于使用多源数据训练大规模状态估计模型，目标是实现跨不同维度系统的泛化。我们希望本文贡献能够为解决广泛真实应用中的状态估计挑战奠定基础。

致谢

本研究得到清华-亿铸未来 EAI 计算与感知联合研究中心支持。

参考文献

- [1] Timothy D Barfoot. *State estimation for robotics*. Cambridge University Press, 2024.
- [2] Kevin Course and Prasanth B Nair. “State estimation of a physical system with unknown governing equations”. In: *Nature* 622.7982 (2023), pp. 261–267.
- [3] Denis Dochain. “State and parameter estimation in chemical and biochemical processes: a tutorial”. In: *Journal of process control* 13.8 (2003), pp. 801–818.
- [4] Jinlei Zhang et al. “Physics-informed deep learning for traffic state estimation based on the traffic flow model and computational graph method”. In: *Information Fusion* 101 (2024), p. 101971.
- [5] Simo Särkkä and Lennart Svensson. *Bayesian filtering and smoothing*. Vol. 17. Cambridge university press, 2023.
- [6] Kazufumi Ito and Kaiqi Xiong. “Gaussian filters for nonlinear filtering problems”. In: *IEEE transactions on automatic control* 45.5 (2000), pp. 910–927.
- [7] Zhe Chen et al. “Bayesian filtering: From Kalman filters to particle filters, and beyond”. In: *Statistics* 182.1 (2003), pp. 1–69.
- [8] Hamed H Afshari, S Andrew Gadsden, and S Habibi. “Gaussian filters for parameter and state estimation: A general review of theory and recent trends”. In: *Signal Processing* 135 (2017), pp. 218–238.
- [9] Ondřej Straka, Jindřich Duník, and Victor Elvira. “Importance Gauss-Hermite Gaussian Filter for Models with Non-Additive Non-Gaussian Noises”. In: *2021 IEEE 24th International Conference on Information Fusion (FUSION)*. IEEE. 2021, pp. 1–7.

- [10] Peter Jan Van Leeuwen et al. “Particle filters for high-dimensional geoscience applications: A review”. In: *Quarterly Journal of the Royal Meteorological Society* 145.723 (2019), pp. 2335–2365.
- [11] Guy Revach et al. “KalmanNet: Neural network aided Kalman filtering for partially known dynamics”. In: *IEEE Transactions on Signal Processing* 70 (2022), pp. 1532–1547.
- [12] Yujie Tang et al. “Reinforcement learning compensated extended Kalman filter for attitude estimation”. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2021, pp. 6854–6859.
- [13] Gwanghyeon Ji et al. “Concurrent training of a control policy and a state estimator for dynamic and robust legged locomotion”. In: *IEEE Robotics and Automation Letters* 7.2 (2022), pp. 4630–4637.
- [14] Pragyan Dahal et al. “RobustStateNet: Robust ego vehicle state estimation for Autonomous Driving”. In: *Robotics and Autonomous Systems* 172 (2024), p. 104585.
- [15] Michael Ahn et al. “Do as i can, not as i say: Grounding language in robotic affordances”. In: *arXiv preprint arXiv:2204.01691* (2022).
- [16] Octo Model Team et al. “Octo: An open-source generalist robot policy”. In: *arXiv preprint arXiv:2405.12213* (2024).
- [17] Anthony Brohan et al. “Rt-1: Robotics transformer for real-world control at scale”. In: *arXiv preprint arXiv:2212.06817* (2022).
- [18] Anthony Brohan et al. “Rt-2: Vision-language-action models transfer web knowledge to robotic control”. In: *arXiv preprint arXiv:2307.15818* (2023).
- [19] Moo Jin Kim et al. “OpenVLA: An Open-Source Vision-Language-Action Model”. In: *arXiv preprint arXiv:2406.09246* (2024).
- [20] Abby O’Neill et al. “Open x-embodiment: Robotic learning datasets and rt-x models”. In: *arXiv preprint arXiv:2310.08864* (2023).
- [21] Thomas Kailath, Ali H Sayed, and Babak Hassibi. *Linear estimation*. Prentice Hall, 2000.
- [22] Graham C Goodwin et al. “Lagrangian duality between constrained estimation and control”. In: *Automatica* 41.6 (2005), pp. 935–944.
- [23] Emanuel Todorov. “General duality between optimal control and estimation”. In: *2008 47th IEEE conference on decision and control*. IEEE. 2008, pp. 4286–4292.
- [24] Jin Won Kim. “Duality for nonlinear filtering”. In: *arXiv preprint arXiv:2207.07709* (2022).
- [25] Rudolph Emil Kalman. “A new approach to linear filtering and prediction problems”. In: (1960).
- [26] Angelo Alessandri, Marco Baglietto, and Giorgio Battistelli. “Moving-horizon state estimation for non-linear discrete-time systems: New stability results and approximation schemes”. In: *Automatica* 44.7 (2008), pp. 1753–1765.
- [27] Josh Achiam et al. “Gpt-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [28] Yong Liu et al. “Autotimes: Autoregressive time series forecasters via large language models”. In: *arXiv preprint arXiv:2402.02370* (2024).
- [29] Yong Liu et al. “Timer: Generative Pre-trained Transformers Are Large Time Series Models”. In: *Forty-first International Conference on Machine Learning*. 2024.
- [30] Brian D Hassard, Nicholas D Kazarinoff, and Yieh-Hei Wan. *Theory and applications of Hopf bifurcation*. Vol. 41. CUP Archive, 1981.
- [31] Shukang Yin et al. “A survey on multimodal large language models”. In: *arXiv preprint arXiv:2306.13549* (2023).

- [32] Danny Driess et al. “Palm-e: An embodied multimodal language model”. In: *arXiv preprint arXiv:2303.03378* (2023).
- [33] Jiaming Han et al. “Onellm: One framework to align all modalities with language”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, pp. 26584–26595.
- [34] Mohinder S Grewal and Angus P Andrews. “Applications of Kalman filtering in aerospace 1960 to the present [historical perspectives]”. In: *IEEE Control Systems Magazine* 30.3 (2010), pp. 69–78.
- [35] Gerald L Smith, Stanley F Schmidt, and Leonard A McGee. *Application of statistical filter theory to the optimal estimation of position and velocity on board a circumlunar vehicle*. Vol. 135. National Aeronautics and Space Administration, 1962.
- [36] Simon J Julier, Jeffrey K Uhlmann, and Hugh F Durrant-Whyte. “A new approach for filtering nonlinear systems”. In: *Proceedings of 1995 American Control Conference-ACC’95*. Vol. 3. IEEE. 1995, pp. 1628–1632.
- [37] Jun S Liu and Rong Chen. “Sequential Monte Carlo methods for dynamic systems”. In: *Journal of the American statistical association* 93.443 (1998), pp. 1032–1044.
- [38] Pavel Sakov and Peter R Oke. “Implications of the form of the ensemble transformation in the ensemble square root filters”. In: *Monthly Weather Review* 136.3 (2008), pp. 1042–1053.
- [39] Xiaoyong Ni, Guy Revach, and Nir Shlezinger. “Adaptive Kalmannet: Data-Driven Kalman Filter with Fast Adaptation”. In: *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2024, pp. 5970–5974.
- [40] S Hochreiter. “Long Short-term Memory”. In: *Neural Computation MIT-Press* (1997).
- [41] Binh Tang and David S Matteson. “Probabilistic transformer for time series analysis”. In: *Advances in Neural Information Processing Systems* 34 (2021), pp. 23592–23608.
- [42] Tian Zhou et al. “One fits all: Power general time series analysis by pretrained lm”. In: *Advances in neural information processing systems* 36 (2023), pp. 43322–43355.
- [43] Nate Gruver et al. “Large language models are zero-shot time series forecasters”. In: *Advances in Neural Information Processing Systems* 36 (2024).
- [44] Ming Jin et al. “Time-llm: Time series forecasting by reprogramming large language models”. In: *arXiv preprint arXiv:2310.01728* (2023).
- [45] Patrick N Raanes, Yumeng Chen, and Colin Grudzien. “DAPPER: data assimilation with Python: a package for experimental research”. In: *Journal of Open Source Software* 9.94 (2024), p. 5150.
- [46] Gerardo Duran-Martin et al. “Outlier-robust Kalman Filtering through Generalised Bayes”. In: *arXiv preprint arXiv:2405.05646* (2024).
- [47] Evgeni E SEL’KOV. “Self-Oscillations in Glycolysis 1. A Simple Kinetic Model”. In: *European Journal of Biochemistry* 4.1 (1968), pp. 79–86.
- [48] Steven L Brunton, Joshua L Proctor, and J Nathan Kutz. “Discovering governing equations from data by sparse identification of nonlinear dynamical systems”. In: *Proceedings of the national academy of sciences* 113.15 (2016), pp. 3932–3937.
- [49] RB Levien and SM Tan. “Double pendulum: An experiment in chaos”. In: *American Journal of Physics* 61 (1993), pp. 1038–1038.
- [50] Edward N Lorenz. “Predictability: A problem partly solved”. In: *Proc. Seminar on predictability*. Vol. 1. 1. Reading. 1996.

- [51] Gabriele Vissio and Valerio Lucarini. “Mechanics and thermodynamics of a new minimal model of the atmosphere”. In: *The European Physical Journal Plus* 135.10 (2020), pp. 1–21.
- [52] Soojin Roh et al. “Observation quality control with a robust ensemble Kalman filter”. In: *Monthly Weather Review* 141.12 (2013), pp. 4414–4428.
- [53] Jacob R Gardner et al. “Bayesian optimization with inequality constraints.” In: *ICML*. Vol. 2014. 2014, pp. 937–945.
- [54] Hugo Touvron et al. “Llama: Open and efficient foundation language models”. In: *arXiv preprint arXiv:2302.13971* (2023).
- [55] Jared Kaplan et al. “Scaling laws for neural language models”. In: *arXiv preprint arXiv:2001.08361* (2020).
- [56] Jesse Dodge et al. “Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping”. In: *arXiv preprint arXiv:2002.06305* (2020).
- [57] Edward J Hu et al. “Lora: Low-rank adaptation of large language models”. In: *arXiv preprint arXiv:2106.09685* (2021).
- [58] Mingtian Tan et al. “Are language models actually useful for time series forecasting?” In: *arXiv preprint arXiv:2406.16964* (2024).
- [59] Alberto Carrassi et al. “Data assimilation in the geosciences: An overview of methods, issues, and perspectives”. In: *Wiley Interdisciplinary Reviews: Climate Change* 9.5 (2018), e535.
- [60] John C Butcher. “Implicit runge-kutta processes”. In: *Mathematics of computation* 18.85 (1964), pp. 50–64.
- [61] Xinshuo Weng et al. “Ab3dmot: A baseline for 3d multi-object tracking and new evaluation metrics”. In: *arXiv preprint arXiv:2008.08063* (2020).
- [62] R OpenAI. “Gpt-4 technical report. arxiv 2303.08774”. In: *View in Article* 2.5 (2023).
- [63] Adam Paszke et al. “Pytorch: An imperative style, high-performance deep learning library”. In: *Advances in neural information processing systems* 32 (2019).
- [64] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018.
- [65] Ilya Loshchilov, Frank Hutter, et al. “Fixing weight decay regularization in adam”. In: *arXiv preprint arXiv:1711.05101* 5 (2017).

NeurIPS 论文检查清单

该检查清单旨在鼓励负责任机器学习研究中的最佳实践，覆盖可复现性、透明度、研究伦理与社会影响等问题。请不要删除检查清单：不包含检查清单的论文将被 desk reject。检查清单应放在参考文献之后，并跟随可选补充材料。检查清单不计入页数限制。

请仔细阅读检查清单指南，以了解如何回答这些问题。对于清单中的每个问题：

- 应回答 [是]、[否] 或 [不适用]。
- [不适用] 表示该问题对这篇论文不适用，或相关信息不可用。
- 请在答案之后立即给出简短的 1-2 句理由，即使答案为 NA 也需要说明。

检查清单答案是论文提交的组成部分。它们对审稿人、area chair、senior area chair 和伦理审查者可见。作者在最终修订版本中也需要包含它，最终版本会随论文一起发布。审稿人会检查清单作为评价因素之一。虽然 [是] 通常优于 [否]，但只要给出适当理由，回答 [否] 完全可以接受；回答 [否] 或 [不适用] 通常不构成拒稿理由。虽然问题以二元方式表述，但真实答案往往更细腻；请使用最佳判断，并用理由展开说明。支持证据可以出现在正文或附录提供的补充材料中。若某问题回答 [是]，请在理由中指出相关材料位于哪些章节。

重要：请删除本说明块，但保留章节标题“NeurIPS Paper Checklist”；保留下面的检查清单小节标题、问题/答案和指南；不要修改问题，并只使用所提供的宏来回答。

1. 声明

问题：摘要和引言中提出的主要声明是否准确反映了论文的贡献和范围？

答案：[是]

理由：摘要和引言中的主要声明准确反映了论文的贡献和范围。

指南：若摘要和引言不包含论文声明，则答案为 NA。摘要和/或引言应清楚陈述论文声明，包括贡献、重要假设和限制。声明应与理论和实验结果匹配，并反映结果可望泛化到其他设置的程度。只要清楚说明这些目标尚未由论文实现，包含愿景式目标作为动机是可以的。

2. 局限性

问题：论文是否讨论了作者所完成工作的局限性？

答案：[是]

理由：本文工作的局限性在第 6 节中讨论。

指南：若论文没有局限性，则答案为 NA；若论文存在局限性但没有讨论，则答案为 No。作者被鼓励在论文中创建单独的“Limitations”章节。论文应指出任何强假设，以及结果对这些假设被违反时的鲁棒性。作者应反思声明的范围，例如方法是否只在少量数据集或少量运行上测试；也应讨论影响方法性能的因素、算法计算效率及其如何随数据集规模扩展。若适用，作者还应讨论方法在隐私和公平问题上的可能限制。

3. 理论假设与证明

问题：对于每个理论结果，论文是否提供了完整假设集合以及完整且正确的证明？

答案：[不适用]

理由：本文不包含理论结果。

指南：若论文不包含理论结果，则答案为 NA。论文中的所有定理、公式和证明都应编号并交叉引用。任何定理陈述中的所有假设都应清楚写出或引用。证明可以出现在正文或补充材料中；若出现在补充材料，鼓励作者在正文中给出简短证明草图。正文中的非正式证明应由附录或补充材料中的正式证明补充。

4. 实验结果可复现性

问题：论文是否充分披露了复现主要实验结果所需的全部信息，达到其影响主要声明和/或结论的程度，而无论代码和数据是否提供？

答案: [是]

理由: 我们在第 5 节提供了复现主要实验结果所需的所有必要细节。

指南: 若论文不包含实验, 则答案为 NA。若论文包含实验, 回答 No 会被审稿人认为较差; 无论代码和数据是否提供, 可复现性都很重要。根据贡献类型, 可复现性可通过完整描述架构、提供同一数据集上的复制路径、提供模型访问、释放 checkpoint 或其他合适方式实现。NeurIPS 不要求发布代码, 但要求提交提供合理的可复现路径。

5. 数据和代码开放访问

问题: 论文是否提供数据和代码的开放访问, 并在补充材料中给出足以忠实复现主要实验结果的说明?

答案: [是]

理由: 我们的实验数据和代码已公开, 可通过 <https://github.com/Anonymous-User-2367/LLM-Filter> 访问。

指南: 若论文不包含需要代码的实验, 则答案为 NA。请参见 NeurIPS 代码和数据提交指南。虽然我们鼓励发布代码和数据, 但理解这并不总是可能, 因此 No 是可接受答案。说明应包含复现实验所需的精确命令和环境, 并说明数据访问、准备、原始数据、预处理数据、中间数据和生成数据等。作者应提供脚本以复现所提方法和 baseline 的所有实验结果; 若只有部分实验可复现, 应说明哪些省略以及原因。

6. 实验设置/细节

问题: 论文是否说明了理解结果所需的所有训练和测试细节, 例如数据划分、超参数、选择方式、优化器类型等?

答案: [是]

理由: 论文在附录 D 和附录 E.4 中说明了所有训练与测试细节以及超参数分析。

指南: 若论文不包含实验, 则答案为 NA。实验设置应在论文核心部分呈现到足以理解结果和评价结果的详细程度。完整细节可随代码、附录或补充材料提供。

7. 实验统计显著性

问题: 论文是否报告了适当且正确定义的误差条, 或其他关于实验统计显著性的合适信息?

答案: [是]

理由: 论文展示了我们实验得到的 RMSE 结果。

指南: 若论文不包含实验, 则答案为 NA。若支持论文主要声明的实验结果附带误差条、置信区间或统计显著性检验, 作者应回答 Yes。应清楚说明误差条捕获的变异因素、计算方式、所作假设, 以及误差条表示标准差还是均值标准误。若使用表格或图形报告误差条, 应在正文解释计算方式并引用相应图表。

8. 实验计算资源

问题: 对于每个实验, 论文是否提供了复现实验所需计算资源的充分信息, 包括计算 worker 类型、内存和执行时间?

答案: [是]

理由: 附录 D 提供了充分的计算资源信息。

指南: 若论文不包含实验, 则答案为 NA。论文应说明 CPU 或 GPU、内部集群或云提供商等计算 worker 类型, 并包含相关内存和存储信息。论文应提供各单独实验运行所需计算量, 并估计总计算量。论文还应披露完整研究项目是否需要比论文报告实验更多的计算, 例如未进入论文的预备或失败实验。

9. 伦理准则

问题: 论文中的研究是否在各方面都符合 NeurIPS Code of Ethics(<https://neurips.cc/public/EthicsGuidelines>)?

答案: [是]

理由: 本文提出的研究在所有方面都完全符合 NeurIPS Code of Ethics。

指南: 若作者没有查看 NeurIPS Code of Ethics, 则答案为 NA。若回答 No, 应解释需要偏离伦理准则的特殊情况。作者应确保保持匿名, 例如避免因所在司法辖区法律或规定而产生的特殊信息暴露。

10. 更广泛影响

问题：论文是否讨论了所完成工作的潜在正面社会影响和负面社会影响？

答案：[是]

理由：本文工作的社会影响在第 6 节中讨论。

指南：若该工作没有社会影响，则答案为 NA。若回答 NA 或 No，作者应解释为何工作没有社会影响，或为何论文未处理社会影响。负面社会影响的例子包括恶意或非预期使用、公平性、隐私和安全问题。许多论文属于基础研究，不必绑定具体部署；但若存在直接通向负面应用的路径，作者应指出。作者应考虑技术按预期正确使用、按预期使用但给出错误结果、以及有意或无意滥用时可能产生的伤害。

11. 保障措施

问题：论文是否描述了为负责任发布具有高滥用风险的数据或模型而设置的保障措施，例如预训练语言模型、图像生成器或抓取数据集？

答案：[不适用]

理由：本文不造成此类风险。

指南：若论文不造成此类风险，则答案为 NA。具有高滥用或双重用途风险的发布模型应附带必要保障，以允许受控使用，例如要求用户遵守使用指南或访问限制，或实现安全过滤器。从互联网抓取的数据集可能带来安全风险，作者应说明如何避免发布不安全图像。我们承认有效保障具有挑战性，许多论文并不需要这一点，但鼓励作者认真考虑。

12. 已有资产许可

问题：论文中使用的已有资产（例如代码、数据、模型）的创建者或原始所有者是否得到适当署名，其许可和使用条款是否被明确提及并恰当遵守？

答案：[是]

理由：是的，资产的创建者和原始所有者均得到适当署名，相关许可和使用条款也被明确提及并遵守。

指南：若论文不使用已有资产，则答案为 NA。作者应引用产生代码包或数据集的原始论文，并说明所用资产版本，可能时包含 URL。每项资产应包含许可证名称。对于来自特定来源的抓取数据，应提供该来源的版权和服务条款。若发布资产，应在包中提供许可证、版权信息和使用条款。

13. 新资产

问题：论文引入的新资产是否有良好文档，并且文档是否随资产一起提供？

答案：[不适用]

理由：本文不发布新资产。

指南：若论文不发布新资产，则答案为 NA。研究者应通过结构化模板在提交中沟通数据集、代码或模型细节，包括训练、许可和限制等。论文应讨论是否以及如何获得被使用资产相关人员的同意。提交时应记得匿名化资产（若适用）。

14. 众包与人类受试者研究

问题：对于众包实验和人类受试者研究，论文是否包含给参与者的完整说明文本和截图（若适用），以及补偿细节（若有）？

答案：[不适用]

理由：本文不涉及众包，也不涉及人类受试者研究。

指南：若论文不涉及众包或人类受试者研究，则答案为 NA。将这些信息包含在补充材料中是可以的；但若论文主要贡献涉及人类受试者，则正文应尽可能包含详细信息。根据 NeurIPS Code of Ethics，参与数据收集、整理或其他劳动的工作人员应至少获得数据收集者所在国家的最低工资。

15. 人类受试者研究的机构审查委员会（IRB）批准或等价批准

问题：论文是否描述了研究参与者可能承担的潜在风险、这些风险是否向受试者披露，以及是否获得机构审查委员会（IRB）批准或根据所在国家/机构要求的等价批准/审查？

答案: [不适用]

理由: 本文不涉及众包, 也不涉及人类受试者研究。

指南: 若论文不涉及众包或人类受试者研究, 则答案为 NA。不同国家对人类受试者研究的 IRB 批准要求可能不同; 若获得 IRB 批准, 论文应清楚说明。初始提交时, 不应包含任何破坏匿名的信息。

16. LLM 使用声明

问题: 若 LLM 是本研究核心方法中的重要、原创或非标准组件, 论文是否描述了 LLM 的使用? 注意, 如果 LLM 仅用于写作、编辑或格式化, 且不影响核心方法、科学严谨性或原创性, 则无需声明。

答案: [不适用]

理由: 本研究中的核心方法开发不涉及将 LLM 作为任何重要、原创或非标准组件。

指南: 若核心方法开发不涉及 LLM 作为重要、原创或非标准组件, 则答案为 NA。请参见 NeurIPS LLM 政策 (<https://neurips.cc/Conferences/2025/LLM>), 了解哪些内容应或不应描述。

A 贝叶斯滤波器

本节介绍经典贝叶斯滤波器，包括 Kalman filter (KF)、Extended Kalman filter (EKF)、Ensemble Kalman filter (EnKF) 和 Particle filter (PF)。关于高级变体的详细描述，建议读者参考第 5 节中引用的相关文献。

A.1 Kalman Filter

假设 SSM 是线性且高斯的：

$$\begin{aligned}\mathbf{x}_{t+1} &= \mathbf{F}_t \mathbf{x}_t + \boldsymbol{\xi}_t, \\ \mathbf{y}_t &= \mathbf{H}_t \mathbf{x}_t + \boldsymbol{\zeta}_t,\end{aligned}$$

其中 $\boldsymbol{\xi}_t \sim \mathcal{N}(\boldsymbol{\xi}_t | 0, \mathbf{Q}_t)$ 和 $\boldsymbol{\zeta}_t \sim \mathcal{N}(\boldsymbol{\zeta}_t | 0, \mathbf{R}_t)$ 表示零均值高斯噪声项，其协方差矩阵分别为 $\mathbf{Q}_t \in \mathbb{R}^{M \times M}$ 和 $\mathbf{R}_t \in \mathbb{R}^{N \times N}$ 。初始状态为高斯分布，给定为 $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{x}_0 | \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$ ，其中均值 $\boldsymbol{\mu}_0 \in \mathbb{R}^M$ 和协方差 $\boldsymbol{\Sigma}_0 \in \mathbb{R}^{M \times M}$ 已知。根据 Kalman filter [25]，先验预测分布和后验分布均保持为高斯分布：

$$\begin{aligned}p(\mathbf{x}_t | \mathbf{y}_{1:t-1}) &= \mathcal{N}(\mathbf{x}_t | \boldsymbol{\mu}_{t|t-1}, \boldsymbol{\Sigma}_{t|t-1}), \\ p(\mathbf{x}_t | \mathbf{y}_{1:t}) &= \mathcal{N}(\mathbf{x}_t | \boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t).\end{aligned}$$

先验预测分布的均值和协方差记为：

$$\begin{aligned}\boldsymbol{\mu}_{t|t-1} &= \mathbf{F}_t \boldsymbol{\mu}_{t-1}, \\ \boldsymbol{\Sigma}_{t|t-1} &= \mathbf{F}_t \boldsymbol{\Sigma}_{t-1} \mathbf{F}_t^\top + \mathbf{Q}_t,\end{aligned}$$

后验均值和协方差为：

$$\begin{aligned}\boldsymbol{\Sigma}_t^{-1} &= \boldsymbol{\Sigma}_{t|t-1}^{-1} + \mathbf{H}_t^\top \mathbf{R}_t^{-1} \mathbf{H}_t, \\ \mathbf{K}_t &= \boldsymbol{\Sigma}_t \mathbf{H}_t^\top \mathbf{R}_t^{-1}, \\ \boldsymbol{\mu}_t &= \boldsymbol{\mu}_{t|t-1} + \mathbf{K}_t (\mathbf{y}_t - \mathbf{H}_t \boldsymbol{\mu}_{t|t-1}).\end{aligned}$$

其中 $\mathbf{K}_t$ 是 Kalman 增益矩阵。

A.2 Extended Kalman Filter

面对非线性系统时，EKF 引入非线性转移模型 $f(\cdot)$ 和观测模型 $h(\cdot)$ 的一阶 Taylor 近似。我们将这些函数的 Jacobian 记为 $\mathbf{F}_x(\cdot)$ 和 $\mathbf{H}_x(\cdot)$ 。于是 EKF 的预测步骤为：

$$\begin{aligned}\boldsymbol{\mu}_{t|t-1} &= f(\boldsymbol{\mu}_{t-1}), \\ \boldsymbol{\Sigma}_{t|t-1} &= \mathbf{F}_x(\boldsymbol{\mu}_{t-1}) \boldsymbol{\Sigma}_{t-1} \mathbf{F}_x(\boldsymbol{\mu}_{t-1})^\top + \mathbf{Q}_t.\end{aligned}$$

更新步骤为：

$$\begin{aligned}\boldsymbol{\Sigma}_t^{-1} &= \boldsymbol{\Sigma}_{t|t-1}^{-1} + \mathbf{H}_x(\boldsymbol{\mu}_{t|t-1})^\top \mathbf{R}_t^{-1} \mathbf{H}_x(\boldsymbol{\mu}_{t|t-1}), \\ \mathbf{K}_t &= \boldsymbol{\Sigma}_t \mathbf{H}_x(\boldsymbol{\mu}_{t|t-1})^\top \mathbf{R}_t^{-1}, \\ \boldsymbol{\mu}_t &= \boldsymbol{\mu}_{t|t-1} + \mathbf{K}_t (\mathbf{y}_t - \mathbf{H}_x(\boldsymbol{\mu}_{t|t-1}) \boldsymbol{\mu}_{t|t-1}).\end{aligned}$$

A.3 Ensemble Kalman Filter

EnKF 被发展为 EKF 在高维和混沌系统中的替代方案，例如天气预报 [59]。EnKF 不显式计算协方差矩阵，而是用 N_e 个粒子 $\hat{\mathbf{x}}_t^i \in \mathbb{R}^M$ 表示 belief state，其中 $i = 1, \dots, N_e$ 。EnKF 的预测步骤使用 (2.3a) 中的转移模型获得 $\hat{\mathbf{x}}_{t|t-1}^i$ 。在更新步骤中，为每个粒子采样预测观测 $\hat{\mathbf{y}}_t^i$ ：

$$\hat{\mathbf{y}}_t^i \sim \mathcal{N}(\hat{\mathbf{y}}_t^i | h(\hat{\mathbf{x}}_{t|t-1}^i), \mathbf{R}_t) \quad i = 1, 2, \dots, N_e.$$

随后，粒子按如下方式更新：

$$\begin{aligned} \mathbf{K}_t &= \text{Cov}(\hat{\mathbf{x}}_{t|t-1}^i, \hat{\mathbf{y}}_t^i) \mathbb{V}(\hat{\mathbf{y}}_t^i)^{-1}, \\ \hat{\mathbf{x}}_t^i &= \hat{\mathbf{x}}_{t|t-1}^i + \mathbf{K}_t(\mathbf{y}_t - \hat{\mathbf{y}}_t^i), \quad i = 1, 2, \dots, N_e, \end{aligned}$$

其中 $\text{Cov}(\cdot, \cdot)$ 和 $\mathbb{V}(\cdot)$ 分别表示协方差和方差。最后，时间 t 的状态估计计算为所有粒子的平均：

$$\hat{\mathbf{x}}_t = \sum_{i=1}^{N_e} \mathbf{x}_t^i.$$

A.4 Particle Filter

PF 是一种基于 sequential importance sampling 的滤波方法 [37]，特点是高精度但计算效率相对较低。PF 的标准实现遵循如下 bootstrap 过程：

(1) **采样**：为所有 N_p 个粒子生成样本；

$$\mathbf{x}_t^i \sim p(\mathbf{x}_t | \mathbf{x}_{t-1}^i), \quad i = 1, 2, \dots, N_p.$$

(2) **加权**：计算所有粒子的权重并归一化；

$$w_t^i \propto p(\mathbf{y}_t | \mathbf{x}_t^i), \quad i = 1, 2, \dots, N_p.$$

(3) **重采样**：根据归一化权重对粒子重采样。

最后，时间 t 的状态估计计算为所有粒子的加权平均：

$$\hat{\mathbf{x}}_t = \sum_{i=1}^{N_p} w_t^i \mathbf{x}_t^i.$$

表 4：实验系统规格。状态空间和观测空间的维度分别记为 M 与 N 。Dataset Size 指从系统收集的数据，格式为（轨迹长度、轨迹数量、状态维度）。Forward Euler 指一阶显式离散化方案，RK4 表示四阶 Runge-Kutta 离散化方案 [60]。

系统	线性	M	N	Dataset Size	Q	R	Δt	离散化	应用领域
Tracking	T	4	2	(200, 100, 6)	$0.1\mathbf{I}$	$10\mathbf{I}$	0.1	Forward Euler	Target Tracking
Selkov	F	2	2	(200, 100, 4)	$\mathbf{I}$	$\mathbf{I}$	0.01	RK4	Glycolysis Process
Oscillator	F	2	2	(200, 100, 4)	$\mathbf{I}$	$\mathbf{I}$	0.01	RK4	Oscillatory Motion
Hopf	F	2	2	(200, 100, 4)	$\mathbf{I}$	$\mathbf{I}$	0.01	RK4	Chemical Reactions
Pendulum	F	4	2	(200, 100, 6)	$\mathbf{I}$	$\mathbf{I}$	0.01	RK4	Physical Dynamics
Lorenz 96	F	72	72	(200, 100, 144)	$\mathbf{I}$	$100\mathbf{I}$	0.01	RK4	Atmospheric Dynamics
VL20	F	72	72	(200, 100, 144)	$\mathbf{I}$	$\mathbf{I}$	0.01	RK4	Atmospheric Dynamics

B 系统描述

本实验使用的系统基于真实世界模型；这些模型在本质上是连续的，并由常微分方程 (ODE) 支配。为把这些连续系统映射到相应的马尔可夫状态空间模型 (SMM)，我们使用 forward Euler 方法和四阶 Runge-Kutta 离散化等方法对其离散化 [60]。对于 (2.1) 中表示为 ξ_t 和 ζ_t 的不确定噪声项，我们假设它们服从标准高斯分布，以保证各模型之间的一致性。

$$\xi_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}), \quad \zeta_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R}),$$

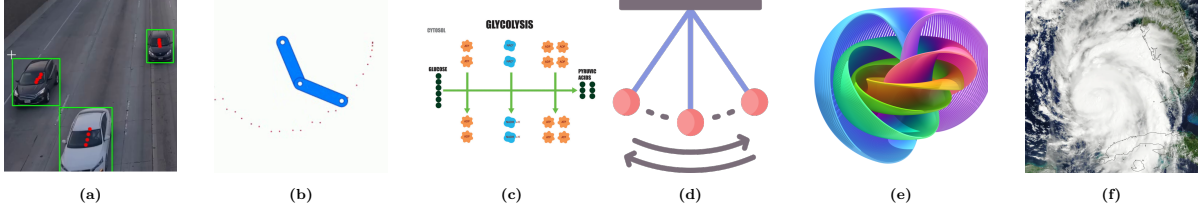


图 8: 实验系统示意。(a) Tracking; (b) Double Pendulum; (c) Selkov; (d) Oscillator; (e) Hopf; (f) Lorenz96 & VL20。

其中 $\mathbf{Q} \in \mathbb{R}^{M \times M}$ 和 $\mathbf{R} \in \mathbb{R}^{N \times N}$ 是协方差。所有系统基本特征的汇总见表 4 和图 8。随后, 我们详细讨论每个系统的具体背景和数学建模。

Tracking Tracking 系统, 也称为 Wiener velocity model, 是一个规范线性高斯系统 [5], 常用于目标跟踪 [61]。状态表示二维空间中运动对象的位置和速度, 写作 $\mathbf{x} = [p_x \ p_y \ v_x \ v_y]^\top$, 其中 p_x 和 p_y 分别表示对象在纵向和横向方向上的位置, v_x 与 v_y 是对应速度。观测是位置分量的直接带噪观测。整体系统可描述为

$$\begin{aligned} \dot{\mathbf{x}} &= \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \mathbf{x}, \\ \mathbf{y}_t &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \mathbf{x}_t + \boldsymbol{\zeta}_t. \end{aligned}$$

Selkov Selkov 系统通过一组两个耦合微分方程来建模糖酵解过程的动力学 [47]。状态向量 $\mathbf{x} = [\theta_1 \ \theta_2]^\top$ 表示参与糖酵解循环的代谢物浓度, 其中 θ_1 与 θ_2 对应关键代谢物浓度。常数 $a = 0.08$ 与 $b = 0.6$ 是系统中与反应速率相关的参数。系统动力学和观测过程由以下方程支配:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = \begin{bmatrix} -\theta_1 + a\theta_2 + \theta_1^2\theta_2 \\ b - a\theta_2 - \theta_1^2\theta_2 \end{bmatrix}, \quad \mathbf{y}_t = \mathbf{x}_t + \boldsymbol{\zeta}_t.$$

Damped Oscillator 阻尼振子指一个物理系统, 其中对象由于存在阻尼力而经历随时间逐渐减小的振荡运动 [48]。在该系统中, 状态向量 $\mathbf{x} = [\theta_1 \ \theta_2]^\top$ 表示振子沿两个轴的位移 θ_1 和 θ_2 。整体系统可描述为:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = \begin{bmatrix} -a\theta_1^3 + b\theta_2^3 \\ -b\theta_1^3 - a\theta_2^3 \end{bmatrix}, \quad \mathbf{y}_t = \mathbf{x}_t + \boldsymbol{\zeta}_t.$$

这里 $a = 0.1$ 和 $b = 2$ 是定义非线性强度的常数, 三次项表示系统中的回复力。

Hopf Bifurcation Hopf 分岔描述这样一种动力系统: 当系统参数 μ 穿过临界阈值时, 系统会从稳定平衡转变为振荡状态并产生自发振荡 [30]。这一现象常见于多种真实系统, 例如化学反应、电路和流体动力学。在该系统中, 状态向量表示为 $\mathbf{x} = [\theta_1 \ \theta_2]^\top$ 。参数 $\mu = 0.5$ 控制系统稳定性, $\omega = 1$ 和 $A = 1$ 分别定义振荡频率和非线性交互强度。整体系统可描述为

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = \begin{bmatrix} \mu\theta_1 + \omega\theta_2 - A\theta_1(\theta_1^2 + \theta_2^2) \\ -\omega\theta_1 + \mu\theta_2 - A\theta_2(\theta_1^2 + \theta_2^2) \end{bmatrix}, \quad \mathbf{y}_t = \mathbf{x}_t + \boldsymbol{\zeta}_t.$$

Double Pendulum Double Pendulum 通常也称为 chaotic pendulum, 是由一个摆末端连接另一个摆构成的非线性动力系统 [49]。状态向量 $\mathbf{x} = [\theta_1 \ \omega_1 \ \theta_2 \ \omega_2]^\top$ 概括了系统构型, 其中 θ_1 与 θ_2 表示角位置, ω_1

与 ω_2 表示两个摆的角速度。系统观测是角度 θ_1 与 θ_2 的带噪观测：

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\omega}_1 \\ \dot{\theta}_2 \\ \dot{\omega}_2 \end{bmatrix} = \begin{bmatrix} \omega_1 \\ \frac{M_2 L_1 \omega_1^2 \sin(D) \cos(D) + M_2 G \sin(\theta_2) \cos(D) + M_2 L_2 \omega_2^2 \sin(D) - (M_1 + M_2) G \sin(\theta_1)}{(M_1 + M_2) L_1 - M_2 L_1 \cos^2(D)} \\ \omega_2 \\ \frac{-M_2 L_2 \omega_2^2 \sin(D) \cos(D) + (M_1 + M_2) G \sin(\theta_1) \cos(D) - (M_1 + M_2) L_1 \omega_1^2 \sin(D) - (M_1 + M_2) G \sin(\theta_2)}{L_1^2 ((M_1 + M_2) L_1 - M_2 L_1 \cos^2(D))} \end{bmatrix},$$

$$\mathbf{y}_t = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \mathbf{x}_t + \boldsymbol{\zeta}_t, \quad D = \theta_2 - \theta_1.$$

这里 $M_1 = 1$ 和 $M_2 = 1$ 是两个摆的质量, $L_1 = 1$ 和 $L_2 = 1$ 表示各自长度, $G = 9.8$ 是支配系统运动的重力常数。

Lorenz96 Lorenz96 系统由 Edward Lorenz 提出 [50], 为大气动力学提供了一个简化但强大的模型, 并常作为混沌理论和气象学中的基石。该系统由一组常微分方程描述, 用于捕捉周期域中大气模态之间的复杂相互作用：

$$\dot{\mathbf{x}}^{(j)} = (\mathbf{x}^{(j+1)} - \mathbf{x}^{(j-2)})\mathbf{x}^{(j-1)} - \mathbf{x}^{(j)} + F, \quad \mathbf{y}_t = \mathbf{x}_t + \boldsymbol{\zeta}_t, \quad j = 1, 2, \dots, M.$$

在该模型中, $\mathbf{x}^{(j)}$ 表示 Lorenz96 系统的第 j 个状态, $F = 8$ 是驱动系统的外部 forcing 项。系统在周期边界条件下运行, 以保证域上的连续性: $\mathbf{x}^{(M+1)} = \mathbf{x}^{(1)}$, $\mathbf{x}^{(0)} = \mathbf{x}^{(M)}$, $\mathbf{x}^{(-1)} = \mathbf{x}^{(M-1)}$, 其中 $M = 72$ 表示状态空间总维数。

Vissio-Lucarini 20 Vissio-Lucarini 20 模型 (VL20) 由 Vissio 和 Lucarini 提出 [51], 是一个耦合系统, 为地球大气动力学提供简约但丰富的表示。通过把 *temperature* 变量整合进既有 Lorenz 框架, VL20 模型能够产生更复杂、更细腻的行为模式：

$$\begin{aligned} \dot{\phi}^{(j)} &= (\phi^{(j+1)} - \phi^{(j-2)})\phi^{(j-1)} - \gamma\phi^{(j)} - \varepsilon\theta^{(j)} + F, \\ \dot{\theta}^{(j)} &= \phi^{(j+1)}\theta^{(j+2)} - \phi^{(j-1)}\theta^{(j-2)} + \varepsilon\phi^{(j)} - \gamma\theta^{(j)} + G, \\ \mathbf{y}_t &= \mathbf{x}_t + \boldsymbol{\zeta}_t, \quad j = 1, 2, \dots, M'. \end{aligned}$$

在该模型中, M' 表示系统中的离散点数量, 状态向量为 $\mathbf{x} = [\phi \ \theta]^\top$, 并有 $M = 2M'$ 个变量。常数 $F = 10, G = 0, \gamma = 1, \varepsilon = 1$ 定义见 [51]。此外, 该系统满足类似 Lorenz96 模型的周期边界条件。

C 原理分析

本节分析大语言模型 (LLM) 与滤波之间的底层联系。如图 9 所示, 我们给出两种过程的简化比较。LLM 基于所有前序 token 预测下一个 token 的概率分布, 而滤波则基于所有过去观测估计当前系统状态的分布。这种根本相似性, 即二者都基于序列历史信息进行预测, 使它们之间能够自然对齐。通过适当对齐, LLM-Filter 可以有效利用 LLM 的推理能力和丰富预训练知识来处理状态估计任务。

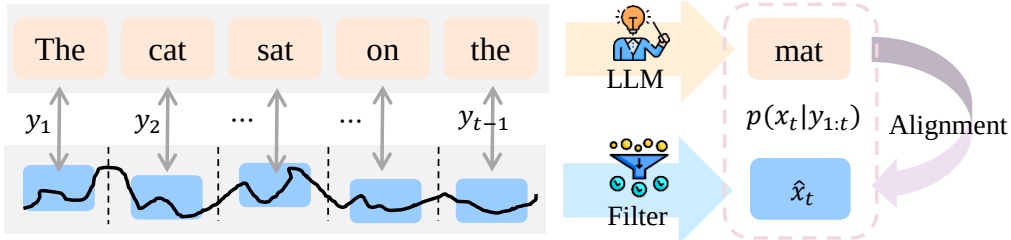


图 9: LLM 与滤波的原理比较。LLM 基于前序 token 预测下一个 token (例如 *mat*), 而滤波器使用过去观测估计当前状态 $\hat{x}_t$ 。

关于更详细的理论解释, 在当前阶段提供它具有挑战性。这一限制源于 LLM 自身理论基础仍是开放研究问题 [27, 62]。因此, LLM 在多样领域中的许多应用, 例如时间序列预测 [28, 42, 44] 和控制系统 [17, 18, 19],

同样缺少严格数学论证。类似地，我们的工作聚焦于经验证据以及语言建模与滤波之间的实际对齐，同时承认未来研究需要更深入的理论理解。

D 实验细节

我们的模型使用 PyTorch [63] 构建，并在同一服务器上充分训练。硬件配置如下：4 个 Intel Xeon Gold 6226R @ 2.90GHz CPU 与 1 个 NVIDIA H800-80G GPU。完整代码公开可访问：<https://github.com/Anonymous-User-2367/LLM-Filter>。

D.1 Benchmark 设置

在线贝叶斯 baseline 方法使用 JAX [64] 实现，并遵循 [46]。EnKF、EnKFI、EnKFS 和 HubEnKF 的粒子数设置为 1000，以平衡效率和准确率 [45]。为公平比较，所有在线贝叶斯方法的超参数在第一次试验中使用 Bayesian Optimization (BO) 包 [53] 选择，参数范围为 (1, 20)。KalmanNet [11] 的实现基于其官方开源仓库。MEstimator [13]、RStateNet [14] 和 ProTran [41] 根据各自论文中的方法实现。表 5 给出了不同模型的参数数量和峰值内存。由于我们冻结 Llama-7B 的核心层，并只训练 ObsEmbedding 与 StateProjection 层，因此区分总参数数量和可训练参数数量。

我们的实验在附录 B 描述的系统上进行。我们从每个系统收集 100 条轨迹，每条包含 200 个时间步，如表 4 所示。为防止数据泄漏，数据集按时间顺序以 7 : 1 : 2 的比例划分为训练集、验证集和测试集。

表 5: 模型参数和峰值内存比较

模型	LLM-Filter (总量/可训练)	MEstimator	RStateNet	ProTran	KalmanNet
Parameter	6.61B / 4.22M	2.12M	7.38M	15.77M	5.20K
Peak memory	27.43GiB	622.51MiB	742.57MiB	854.54MiB	582.93MiB

D.2 实现

在 LLM-Filter 中，只有 ObsEmbedding 和 StateProjection 层通过梯度更新。这些组件用简单 MLP 实现，已有研究表明这种方式可以有效嵌入并投影面向 LLM 的时间序列 [28]。默认 hidden dimension、MLP 层数、batch size、窗口长度 T 和分段长度 L 分别设置为 512、2、16、40 和 20。我们使用 AdamW 优化器 [65]，初始学习率为 10^{-4} ，weight decay 为 10^{-5} ，模型训练 10 个 epoch。除非另有说明，我们使用 LLaMA-7B [54] 作为默认基础 LLM。

D.3 评价指标

评估时，我们使用均方根误差 (RMSE) 作为主要指标，用于评估状态估计准确率 [5]。为了评估效率，我们测量每步推理运行时间，单位为毫秒 (ms/step)。这些指标的计算如下：

$$\text{RMSE} = \sqrt{\frac{\sum_{t=1}^{T_{traj}} \|x_t - \hat{x}_t\|^2}{T_{traj}}}, \quad \text{Runtime} = \frac{\text{Cost Time}}{T_{traj} \cdot N_{test}},$$

其中 T_{traj} 是轨迹长度， N_{test} 表示测试集中的轨迹数量。

E 补充结果

E.1 运行时间

我们在附录 D 描述的同一设备上评估了多种方法的估计效率 (ms/step)。结果见表 6。

表 6: 不同方法的估计运行时间 (ms/step) 比较。

方法	LLM-Filter	EnKF	RStateNet
Selkov	0.8766	2.5095	0.1546
Oscillator	0.8299	2.6815	0.1553
Hopf	0.8245	2.5747	0.1267
Pendulum	1.2447	2.2246	0.0967
Lorenz96	0.9577	6.7377	0.4513
VL20	0.9503	2.9397	0.4521

E.2 泛化估计实验

为评估 LLM-Filter 的泛化能力, 我们开展了广泛评估。首先, 我们评估其在受到不同强度 outlier 影响的系统上的表现, 详见表 7。值得注意的是, 虽然其他方法的性能会随着 outlier 增多而快速恶化, LLM-Filter 的性能损失很小。

我们还测试了经典观测扰动场景 [46], 在该场景中 EnKFI、EnKFS 和 HubEnKF 等鲁棒滤波方法已知表现优异。如表 8 所示, 尽管 LLM-Filter 没有取得最佳结果, 但其性能可与专用鲁棒算法相比。这些发现表明, 即使存在显著 outlier 干扰或系统扰动, LLM-Filter 仍能有效泛化, 为多样且具有挑战性的场景提供可靠解决方案。

此外, 我们在表 9 中提供第 5.2 节讨论的跨系统场景完整实验结果。

表 7: 观测 outlier 场景下的 RMSE 结果。这里 R 表示含 outlier 的真实观测协方差, R_0 表示假定协方差。

方法	R	LLM-Filter	EnKF	EnKFI	EnKFS	HubEnKF
Selkov	R_0	0.4061	<u>0.5978</u>	0.6703	0.6081	0.6676
	$25R_0$	0.6898	1.8626	1.9874	<u>1.6523</u>	2.1998
	$50R_0$	0.7034	2.5496	3.4248	2.6597	<u>2.5318</u>
	$75R_0$	0.7074	3.0952	4.3829	3.1699	<u>2.8636</u>
	$100R_0$	0.7090	3.5625	4.8417	4.3972	<u>2.9850</u>
Oscillator	R_0	0.5247	<u>0.5505</u>	0.5516	0.5548	0.5540
	$25R_0$	0.5946	<u>1.1630</u>	1.2714	1.1674	1.1746
	$50R_0$	0.6044	1.3267	<u>1.3195</u>	1.3575	1.4778
	$75R_0$	0.6083	<u>1.4701</u>	3.1428	1.4757	1.8707
	$100R_0$	0.6110	1.6154	1.6040	<u>1.5793</u>	2.1680
Hopf	R_0	0.5751	<u>0.6322</u>	0.6983	0.6380	0.6982
	$25R_0$	0.7142	0.9946	1.1299	<u>0.9938</u>	1.0976
	$50R_0$	0.7738	<u>1.0549</u>	1.1657	1.2035	1.1155
	$75R_0$	0.8010	<u>1.0898</u>	1.1872	1.5071	1.1313
	$100R_0$	0.8096	<u>1.1146</u>	1.1989	2.1321	1.1483

表 8: 观测扰动场景的 RMSE 结果。

方法	LLM-Filter	EnKF	EnKFI	EnKFS	HubEnKF
Selkov	0.4999	0.9223	0.9500	<u>0.6079</u>	0.6198
Oscillator	0.7451	1.3431	1.3611	<u>0.5543</u>	0.5364
Hopf	0.6827	0.8220	NaN	<u>0.6379</u>	0.7226
Lorenz96	0.9186	7.6225	7.7459	4.5833	<u>3.1399</u>
VL20	0.8876	NaN	7.7777	11.5041	<u>0.9684</u>

表 9: 跨系统场景的 RMSE。

方法	LLM-Filter	LLM-Filter-O	MEstimator	RStateNet	ProTran
Tracking→Pendulum	1.2568	<u>1.6573</u>	3.6957	3.1052	3.1162
Pendulum→Tracking	1.4214	<u>1.8936</u>	2.7407	5.0927	NaN
Oscillator→Hopf	0.6613	<u>2.1125</u>	NaN	3.8645	3.5668
Hopf→Oscillator	0.9069	<u>1.5624</u>	2.5369	2.7542	3.0075

E.3 Scaling 行为

我们通过评估具有多种预训练 LLM 主干的模型，研究 LLM-Filter 在状态估计任务中的 scaling 行为。详细结果见表 10。分析表明，LLM-Filter 遵循 scaling-law 行为：随着模型参数规模增加，平均估计误差趋于下降，从而带来更好的估计性能。

表 10: LLM-Filter 框架中多种 LLM 的 RMSE 结果。

LLM	GPT-2 (124M)	OPT-1.3B	LLaMA-7B	LLaMA-13B
Selkov	1.0434	1.1713	<u>0.6369</u>	0.6200
Oscillator	0.9393	1.0702	<u>0.5753</u>	0.4441
Hopf	0.9611	0.8466	<u>0.8180</u>	0.7000
Lorenz96	0.9727	<u>0.9029</u>	0.9735	0.9147
VL20	0.9931	0.5999	0.8433	<u>0.7623</u>
Average	0.9819	0.9182	<u>0.7694</u>	0.6882

E.4 超参数敏感性

在主实验中，LLM-Filter 的配置见附录 D.2。这里，我们验证 LLM-Filter 对超参数的鲁棒性，包括滑动长度 T 、hidden layer dimension，以及 ObsEmbedding 和 StateProjection 中 MLP 的层数。实验在 Selkov 与 VL20 系统上进行，结果汇总于图 10。整体而言，LLM-Filter 在不同参数设置下表现稳定，说明它对超参数变化大体不敏感。平均来看，窗口长度为 20 或 40 时效果更好。这表明适当选择窗口长度可以取得平衡：较短窗口可能丢失关键信息，而较长窗口可能无法有效捕获局部特征。

此外，LLM-Filter 中的分段长度是一个超参数，用于决定输入 LLM 前把多少个观测分组为一个 token。我们在固定窗口长度为 40 的情况下，使用不同分段长度进行了进一步实验，如表 11 所示。LLM-Filter 在不同分段长度下保持相对稳定的性能，体现出它对该超参数的鲁棒性。

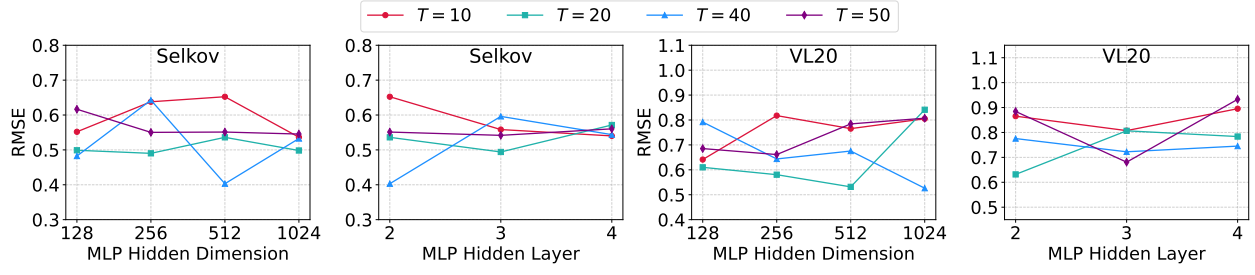


图 10: LLM-Filter 在 Selkov 与 VL20 系统上的超参数敏感性。

表 11: 不同分段长度下的 RMSE 比较

Segment Length	Base (20)	10	5
Selkov	0.4061	0.7024	0.3077
Oscillator	0.5247	0.5829	0.5298
Hopf	0.5751	0.6222	0.5779
Pendulum	0.8348	0.5028	0.8254
Lorenz96	0.9149	0.9200	0.9182
VL20	0.7717	1.0346	1.0221
Average	0.6712	0.7275	0.6969